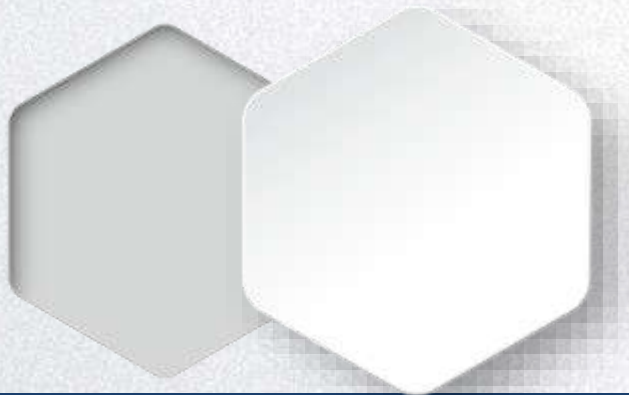




《人工智能与Python程序设计》——PyTorch神经网络



人工智能与Python程序设计 教研组



PyTorch神经网络

提纲



- ☐ 神经网络概述
- ☐ 使用PyTorch搭建神经网络
- ☐ 使用PyTorch训练神经网络
-

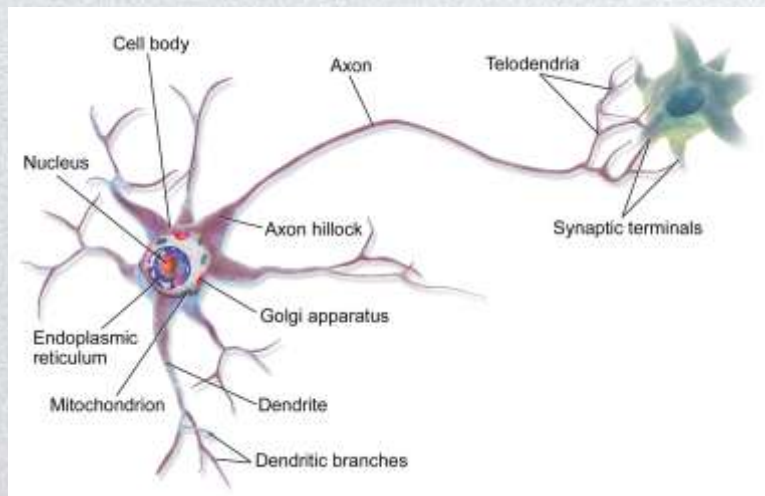


神经网络概述

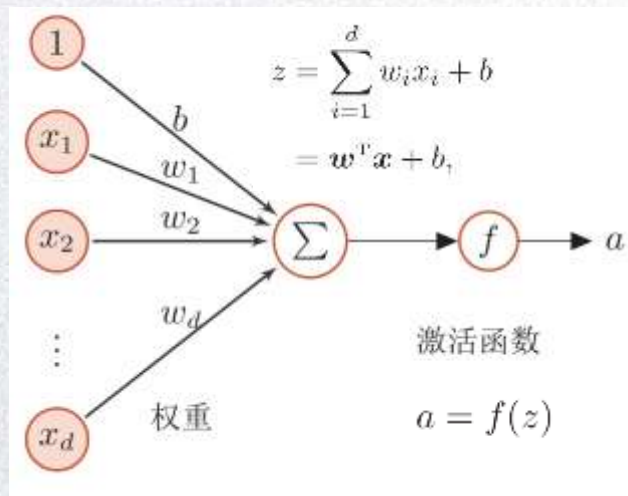
- 神经网络
 - 神经网络可以指向两种
 - 生物神经网络
 - 人工神经网络
 - 生物神经网络
 - 一般指生物的大脑神经元、细胞、触点等组成的网络
 - 用于产生生物的意识，帮助生物进行思考和行动
 - 人工神经网络也简称为神经网络 (Neural Network)
 - 模仿动物神经网络的行为特征，进行分布式、深层信息处理的算法模型

神经网络概述

- 生物神经元 v.s. 人工神经元



单个神经细胞有两种状态：
抑制或者兴奋



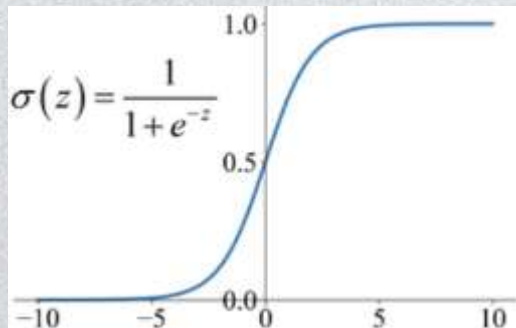
线性模型配上激活函数



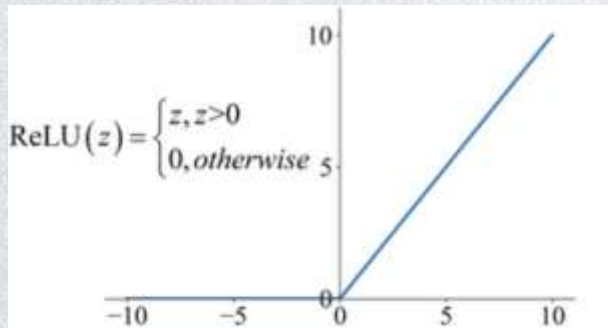
神经网络概述

- 激活函数

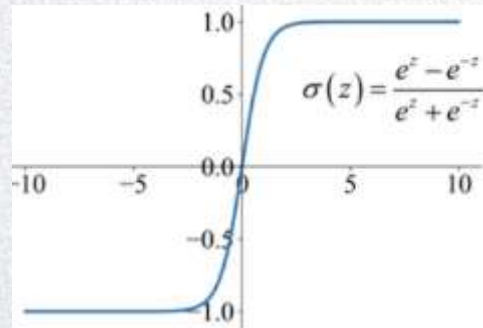
$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$



$$\text{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases} \\ = \max(0, x).$$

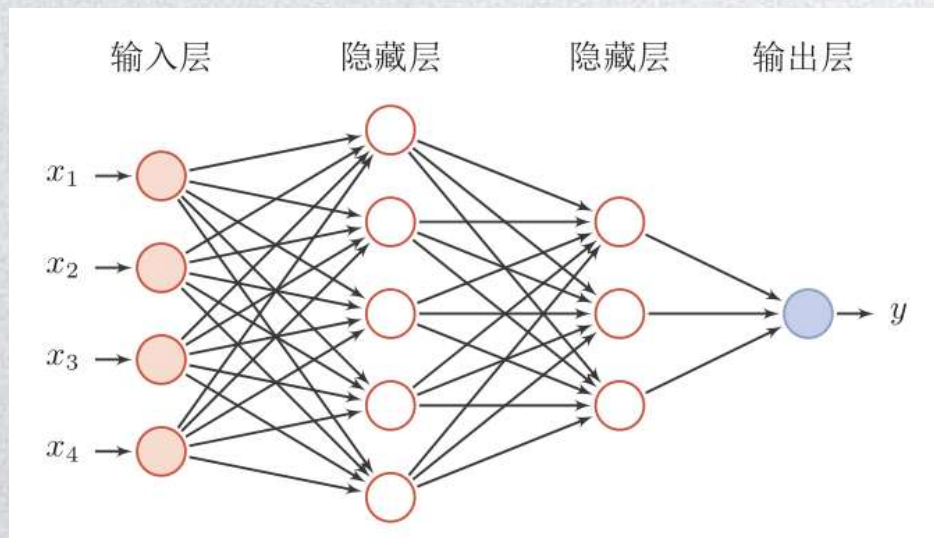


$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$



神经网络概述

- 最常见的网络结构
 - 前馈神经网络
 - 也称为多层感知机 (MLP) 或者全连接网络



$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)},$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)}).$$

记号	含义
L	神经网络的层数
M_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$\mathbf{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$\mathbf{b}^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$\mathbf{z}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入 (净活性值)
$\mathbf{a}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出 (活性值)



神经网络概述

- 神经网络本质上是一种复合函数

$$\hat{y} = f^L (... (f^3(f^2(f^1(x))))))$$

- 两个重要过程

- 前向计算 (Forward computation)

$$\mathbf{x} \rightarrow \mathbf{a}^{(0)} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{a}^{(1)} \rightarrow \mathbf{z}^{(2)} \rightarrow \dots \rightarrow \mathbf{a}^{(L-1)} \rightarrow \mathbf{z}^{(L)} = \hat{\mathbf{y}}$$

- 后向求导 (Back-propagation) [感兴趣的同学阅读注释链接]

$$\delta^l = ((\mathbf{w}^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l)$$

从后回传误差

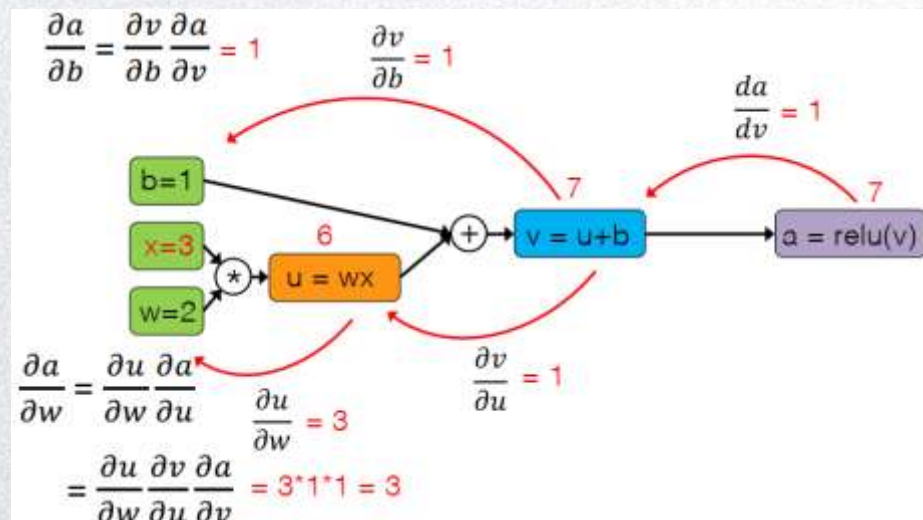
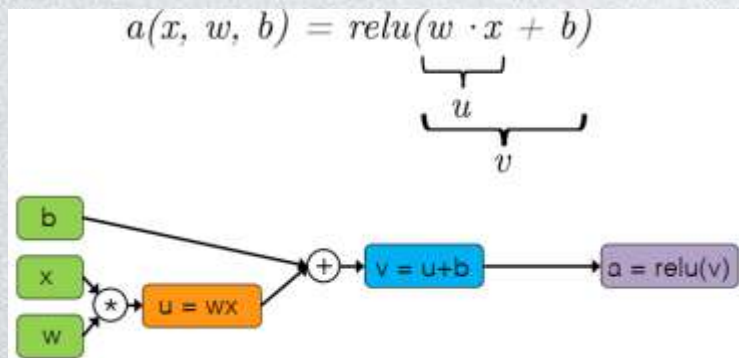
$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l \text{ and } \frac{\partial C}{\partial b_j^l} = \delta_j^l.$$

逐层计算导数

<http://neuralnetworksanddeeplearning.com/chap2.html>

神经网络概述

- 自动求导的本质
 - 构建计算图，通过可达的路径累积计算导数





神经网络概述

- 优化的一般流程

- 规定函数形式，明确如下

- 参数 (需要学习，一般用 w 表示)
 - 输入 (始终给定，一般用 x 表示)
 - 输出 (训练时给定，一般用 y 表示)
 - 预测 (由模型的出来，一般用 $\hat{y}$ 表示)

- 构建关于输出 y 与预测 $\hat{y}$ 之间的损失函数 $L(y, \hat{y})$

- 以参数为待求目标，进行反向求导 $\frac{dL}{dw}$

- 使用梯度下降 (或者其变种) 进行参数的更新:

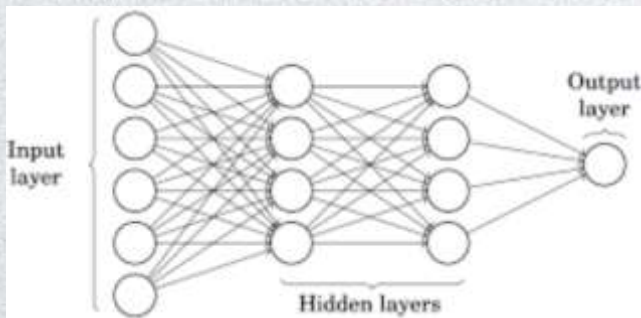
$$w^{new} = w^{old} - lr * \frac{dL}{dw}$$

- 迭代多轮，直至收敛

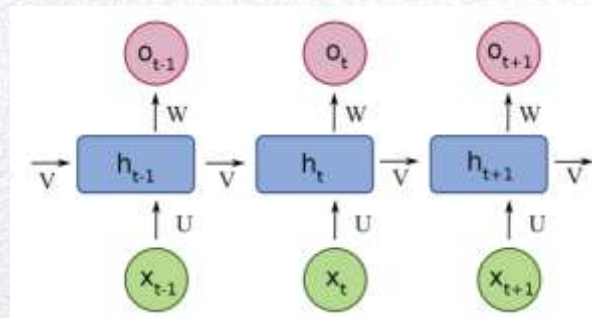
神经网络概述

常见的神经网络

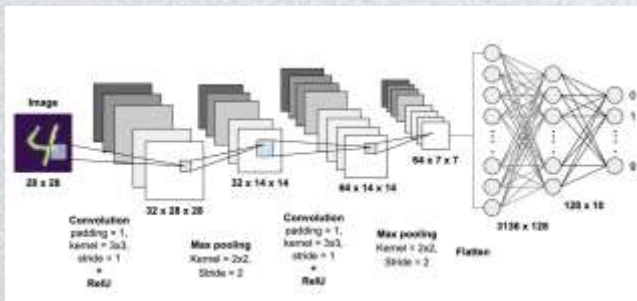
多层感知机 (MLP)



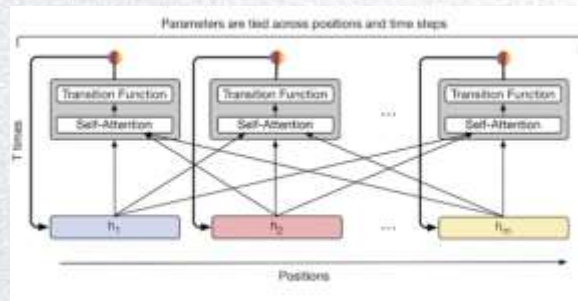
循环神经网络



卷积神经网络



自注意力机制网络



神经网络概述



- 之前的函数
 - 多元线性回归
 - 只有一个线性层
 - $\hat{y} = \mathbf{w}^T \mathbf{x} + b$
 - 多元逻辑回归
 - 只有一个线性层+sigmoid激活函数
 - $\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b)$



PyTorch神经网络

提纲



- ☐ 神经网络概述
- ☐ 使用PyTorch搭建神经网络
- ☐ 使用PyTorch训练神经网络
-



回顾：使用PyTorch实现线性回归模型

- 继承nn.Module，实现线性回归模型

```
class LinearRegression(nn.Module):  
    def __init__(self, in_dim): #构造函数，需要调用nn.Module的构造函数  
        super().__init__()      #等价于nn.Module.__init__()  
        self.w=nn.Parameter(torch.randn(in_dim+1, 1))  
  
    def forward(self, x):  
        x = torch.cat([x, torch.ones((x.shape[0],1))], dim = 1)  
        x = x.matmul(self.w)  
        return x
```

- 自己实现了一个线性层



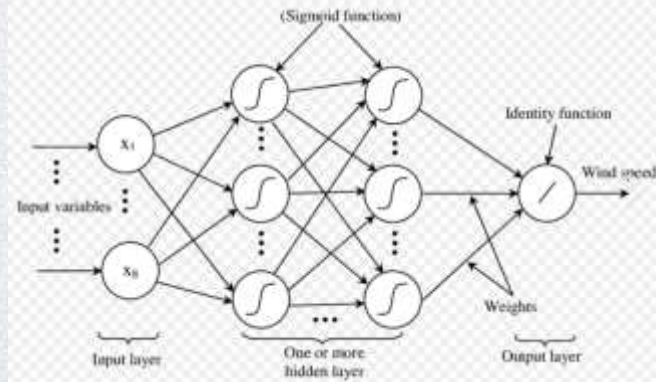
回顾：使用PyTorch实现线性回归模型

- 继承nn.Module，实现线性回归模型：
 - 步骤1：实现__init__方法，初始化线性回归参数，保存到self.w属性中
 - 模型参数需要是nn.Parameter类型的，这样该属性才能被self.parameters()迭代器遍历，进而被optimizer优化和更新
 - 不要忘了调用super().__init__()
 - 步骤2：实现forward方法，计算 $\hat{y} = xw$



目标：实现一个基于MLP的回归模型

- 继承`nn.Module`，实现MLP（Multi-Layer Perceptron）回归模型：
 - 步骤1：实现`__init__`方法
 - 步骤2：实现`forward`方法
- 使用`torch.nn`中自带的神经网络模块实现该模型
 - `nn`包中自带很多常用的神经网络模块
 - 线性层
 - 激活函数
 - 这些模块均继承`nn.Module`
 - 我们常把这些模块称为神经网络中的“层”



线性回归

- 多元线性回归
- $\hat{y} = x w^T + b$

$x_1^{(1)}$	$x_1^{(2)}$	...	$x_1^{(d)}$
$x_2^{(1)}$	$x_2^{(2)}$	...	$x_2^{(d)}$
$\vdots$	$\vdots$		$\vdots$
$x_N^{(1)}$	$x_N^{(2)}$	...	$x_N^{(d)}$

w_1
w_2
$\vdots$
w_d

$+b =$

y_1
y_2
$\vdots$
y_N



单层线性回归

- Pytorch自带线性层/全连接层可以实现单层线性回归

```
class Linear_Model():  
    def __init__(self, in_dim):  
        """  
        创建模型和优化器，初始化线性模型和优化器超参数  
        """  
        self.learning_rate = 0.01  
        self.epochs = 10000  
        self.model = LinearRegression(in_dim) #torch.nn.Linear(in_dim,1)  
        self.optimizer = torch.optim.SGD(self.model.parameters(), lr=self.learning_rate)  
        self.loss_function = torch.nn.MSELoss()
```


PyTorch线性层

- 线性层（全连接层）： $X: N*d, W: d' *d, b: 1*d'$
- $\hat{y}=XW^T + b$

$x_1^{(1)}$	$x_1^{(2)}$	...	$x_1^{(d)}$
$x_2^{(1)}$	$x_2^{(2)}$	...	$x_2^{(d)}$
$\vdots$	$\vdots$		$\vdots$
$x_N^{(1)}$	$x_N^{(2)}$	...	$x_N^{(d)}$

$w_1^{(1)}$	$w_1^{(2)}$	...	$w_1^{(d')}$
$w_2^{(1)}$	$w_2^{(2)}$	...	$w_2^{(d')}$
$\vdots$	$\vdots$		$\vdots$
$w_d^{(1)}$	$w_d^{(2)}$	...	$w_d^{(d')}$

=

$y_1^{(1)}$	$y_1^{(2)}$	...	$y_1^{(d')}$
$y_2^{(1)}$	$y_2^{(2)}$	...	$y_2^{(d')}$
$\vdots$	$\vdots$		$\vdots$
$y_N^{(1)}$	$y_N^{(2)}$	...	$y_N^{(d')}$

+

b_1	b_2	$\vdots$	$b_{d'}$
-------	-------	----------	----------



Pytorch线性回归

- 多目标回归：回归值是多维向量

```
class Linear_Model():
    def __init__(self, in_dim, out_dim):
        """
        创建模型和优化器，初始化线性模型和优化器超参数
        """
        self.learning_rate = 0.01
        self.epoches = 10000
        #self.model = LinearRegression(in_dim) #torch.nn.Linear(in_dim,1)
        self.model = torch.nn.Linear(in_dim,out_dim)
        self.optimizer = torch.optim.SGD(self.model.parameters(), lr=self.learning_rate)
        self.loss_function = torch.nn.MSELoss()
```



PyTorch线性层

- `torch.nn.Linear`(in_features, out_features, **bias=True**)
 - in_features: 输入特征的维数，即输入的张量shape: `[data_size, in_dim]`中的in_dim
 - out_features: 输出数据的维数，即输出的二维张量的形状为`[data_size, out_dim]`，也代表了该全连接层的神经元个数
 - 从输入输出的张量的shape角度来理解，一个输入为`[data_size, in_features]`的张量变换成了`[data_size, out_features]`的输出张量
 - data_size: 输入特征的总数（样本数）
 - Bias: 线性变换 $w^T x + b$ 是否加***b***



PyTorch多个线性层

- Pytorch堆积多个线性层

$$z = f_1(W_1^T x + b_1)$$

$$\hat{y} = f_2(W_2^T z + b_2)$$

```
class MLPRegression(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.linear_hidden_layer = nn.Linear(input_dim, hidden_dim, bias=True)
        self.linear_output_layer = nn.Linear(hidden_dim, 1, bias=True)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        hidden_layer_output = self.sigmoid(self.linear_hidden_layer(x))
        output = self.sigmoid(self.linear_output_layer(hidden_layer_output))
        return output
```



PyTorch多个线性层

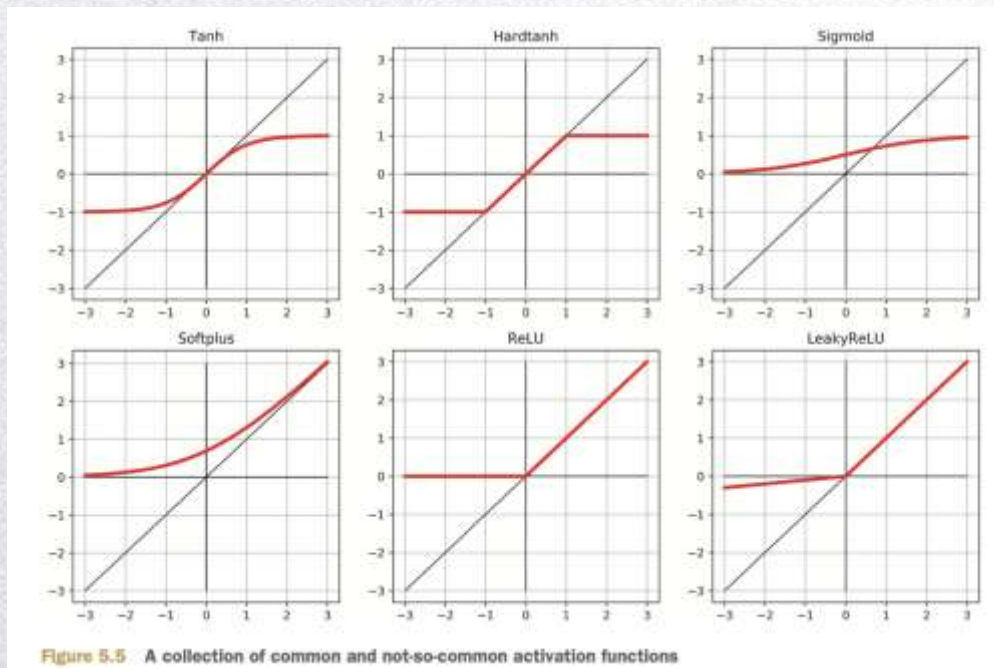
- 在forward方法中：
 - 将self.linear_hidden_layer的输出，输入self.sigmoid激活函数，得到hidden_layer_output
 - 将self.ouput_layer的输出，输入self.sigmoid激活函数，得到output

```
class MLPRegression(nn.Module):  
    def __init__(self, input_dim, hidden_dim):  
        super().__init__()  
        self.linear_hidden_layer = nn.Linear(input_dim, hidden_dim, bias=True)  
        self.linear_output_layer = nn.Linear(hidden_dim, 1, bias=True)  
        self.sigmoid = nn.Sigmoid()  
  
    def forward(self, x):  
        hidden_layer_output = self.sigmoid(self.linear_hidden_layer(x))  
        output = self.sigmoid(self.linear_output_layer(hidden_layer_output))  
        return output
```

- 思考：为什么在两个线性层之间需要有激活函数？

PyTorch中提供的激活函数

- Pytorch为我们提供了常用的激活函数
 - nn.Sigmoid
 - nn.Tanh
 - nn.ReLU
 - ...





问题：MLPRegression的参数是什么？

- 遍历MLPRegression类对象的parameters():

```
def testMLPRegressionModel(in_dim, hidden_dim=2, data_size=2):  
    layer = MLPRegression(in_dim, hidden_dim)  
    input=torch.randn(data_size,in_dim)  
    output=layer(input) #前向传播 执行forward()  
    print(output)  
    for name, parameter in layer.named_parameters():  
        print(name, parameter)  
  
testMLPRegressionModel(3)
```

```
tensor([[0.0971],  
        [0.3525]], grad_fn=<SigmoidBackward>)  
linear_hidden Parameter containing:  
tensor([[ 0.2793,  0.7372],  
        [ 1.3347, -0.9731],  
        [-0.8921, -1.3649],  
        [-1.5682,  1.1941]], requires_grad=True)  
linear_output Parameter containing:  
tensor([[ -2.4742],  
        [-0.9590]], requires_grad=True)
```

- 一个nn.Module的子类（MLPRegression）的parameters()方法能够用来遍历它的所有属性中：
 - nn.Parameter类对象
 - nn.Module子类对象的parameters()方法能够遍历的对象



PyTorch神经网络

提纲



- ☐ 神经网络概述
- ☐ 使用PyTorch搭建神经网络
- ☐ 使用PyTorch训练神经网络
- ☐



如何训练神经网络

- 模型训练和优化的一般流程
 - 规定函数形式，明确如下
 - **参数** (需要学习，一般用 w 表示)
 - **输入** (始终给定，一般用 x 表示)
 - **输出** (训练时给定，一般用 y 表示)
 - **预测** (由模型的出来，一般用 $\hat{y}$ 表示)
 - 构建关于输出 y 与预测 $\hat{y}$ 之间的**损失函数** $L(y, \hat{y})$
 - 以参数为待求目标，进行反向求导 $\frac{dL}{dw}$
 - 使用**梯度下降 (或者其变种)** 进行参数的更新:
$$w^{new} = w^{old} - lr * \frac{dL}{dw}$$
 - 迭代多轮，直至收敛

如何使用PyTorch训练神经网络

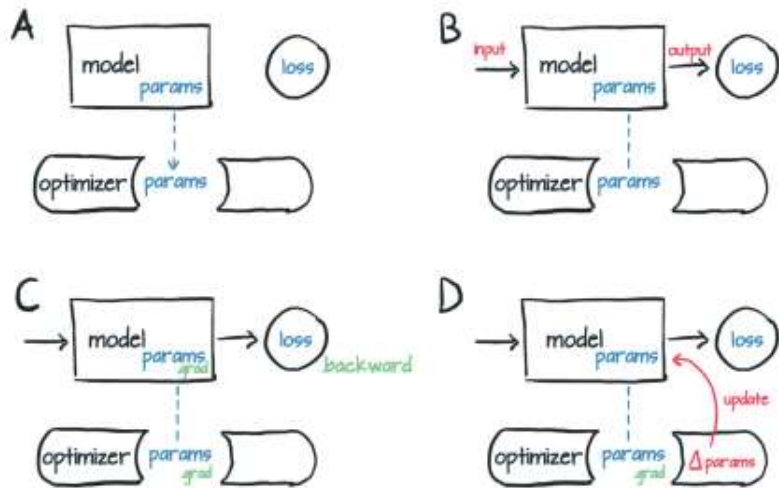


Figure 4.10 Conceptual representation of how an optimizer holds a reference to parameters (A), and after a loss is computed from inputs (B), a call to `.backward` leads to `.grad` being populated on parameter (C). At that point, the optimizer can access `.grad` and compute the parameter updates (D).

```
def train(self, x, y):
    """
    训练模型并保存参数
    输入:
        model_save_path: saved name of model
        x: 训练数据
        y: 回归真值
    返回:
        losses: 所有迭代中损失函数值
    """
    losses = []
    for epoch in range(self.epochs):
        prediction = self.model(x)
        loss = self.loss_function(prediction, y)

        self.optimizer.zero_grad()
        loss.backward()
        self.optimizer.step()

        losses.append(loss.item())

    if epoch % 500 == 0:
        print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

- 我们需要根据不同的任务选择合适的损失函数和优化器



PyTorch中的损失函数

- torch.nn中同样提供了很多常用的损失函数：
 - <https://pytorch.org/docs/stable/nn.html#loss-functions>

Loss Functions

`nn.L1Loss`

`nn.KLDivLoss`

`nn.BCELoss`

`nn.SoftMarginLoss`

Creates a criterion that optimizes a two-class classification logistic loss between input tensor x and target tensor y (containing 1 or -1).

`nn.MSELoss`

`nn.BCEWithLogitsLoss`

`nn.MultiLabelSoftMarginLoss`

Creates a criterion that optimizes a multi-label one-versus-all loss based on max-entropy, between input x and target y of size (N, C) .

`nn.CrossEntropyLoss`

`nn.MarginRankingLoss`

`nn.CosineEmbeddingLoss`

Creates a criterion that measures the loss given input tensors x_1 , x_2 and a Tensor label y with values 1 or -1.

`nn.CTCLoss`

`nn.HingeEmbeddingLoss`

`nn.MultiMarginLoss`

Creates a criterion that optimizes a multi-class classification hinge loss (margin-based loss) between input x (a 2D mini-batch Tensor) and output y (which is a 1D tensor of target class indices, $0 \leq y \leq x.size(1) - 1$).

`nn.NLLLoss`

`nn.MultiLabelMarginLoss`

`nn.TripletMarginLoss`

Creates a criterion that measures the triplet loss given an input tensors x_1 , x_2 , x_3 and a margin with a value greater than 0.

`nn.PoissonNLLLoss`

`nn.GaussianNLLLoss`

`nn.SmoothL1Loss`



PyTorch中的损失函数

- 损失函数 $L(y, \hat{y})$:
 - 衡量模型预测 $\hat{y}$ 和实际真实输出 y 之间的误差
 - 损失函数越小，模型预测越准确
 - 通过最小化训练集上的损失函数，来训练模型
- 如何选择损失函数？
 - 通常我们依据要完成的任务，选择合适的损失函数

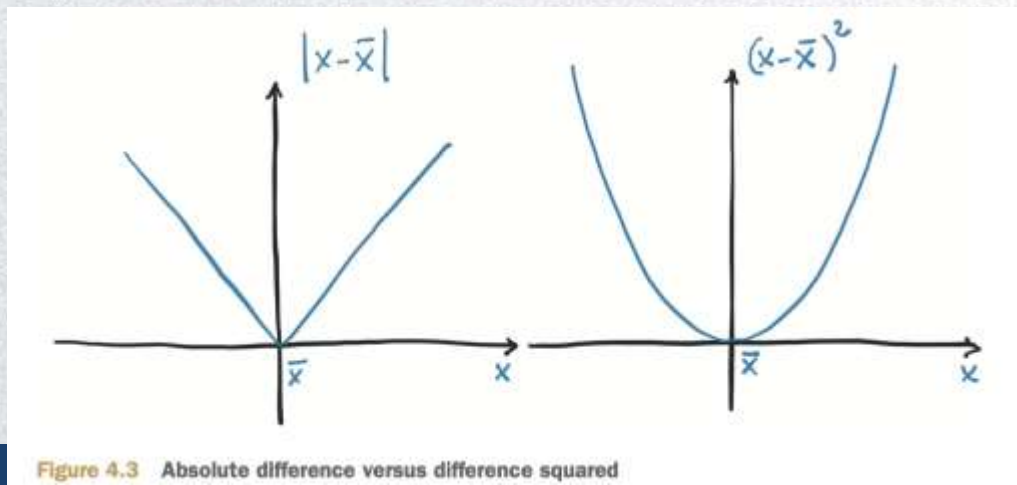


PyTorch中的损失函数

- 常用损失函数:

- 回归模型:

- 均方误差函数 (Mean Squared Error, MSE) : $\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$
 - [nn.MSELoss](#)
 - 平均绝对值误差 (Mean Absolute Error, MAE): $\frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$
 - [nn.L1Loss](#)





PyTorch中的损失函数

- 常用损失函数:

- 二分类模型

- 逻辑回归损失函数: $-\frac{1}{N} \sum_{i=1}^N [y_i \log q_i + (1 - y_i) \log(1 - q_i)]$

- [nn.BCELoss](#)

- 二元交叉熵 (Binary Cross Entropy)

- 多分类模型

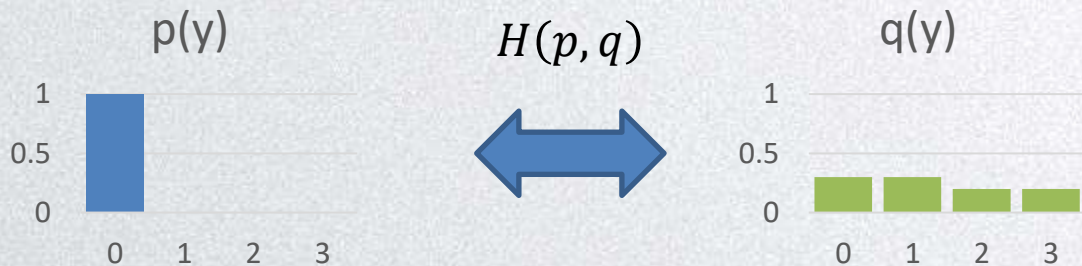
- 交叉熵损失函数:

- [nn.CrossEntropyLoss](#)

$$\text{loss}(x, \text{class}) = -\log \left(\frac{\exp(x[\text{class}])}{\sum_j \exp(x[j])} \right)$$

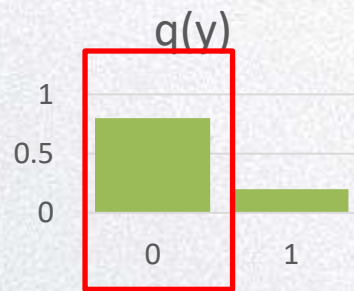
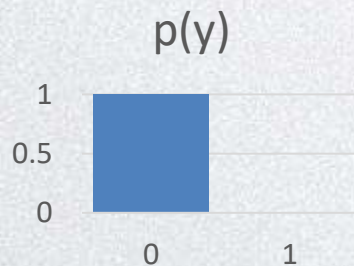
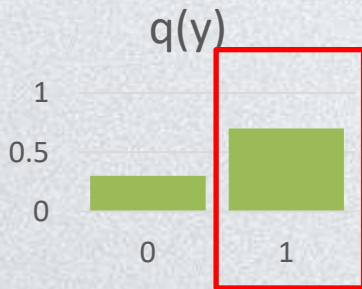
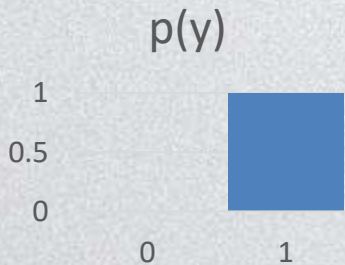
PyTorch中的损失函数

- 交叉熵 (Cross-Entropy):
 - $H(p, q) = -\sum_y p(y) \cdot \log q(y)$
 - 衡量真实分布 $p(y)$ 和预测分布 $q(y)$ 之间的差异
 - 如果 $p(y) = q(y)$, 那么交叉熵 $H(p, q)$ 最小, 且刚好等于 $p(y)$ 的熵



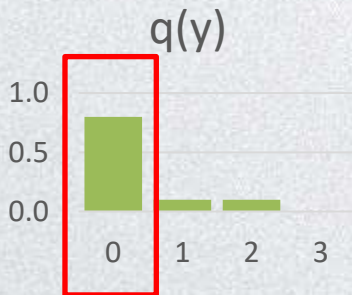
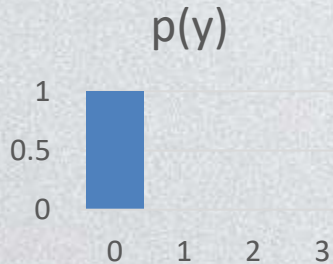
PyTorch中的损失函数

- 二分类: $y \in \{0, 1\}$
 - $H(p, q) = -\sum_y p(y) \cdot \log q(y)$
 $= -\{y \cdot \log q(y = 1) + (1 - y) \cdot \log(1 - q(y = 1))\}$
 - 在所有数据上取平均: $-\frac{1}{N} \sum_{i=1}^N [y_i \ln q_i + (1 - y_i) \ln(1 - q_i)]$
 - 逻辑回归: $q(y = 1) = \frac{1}{1 + e^{-(xw+b)}}$

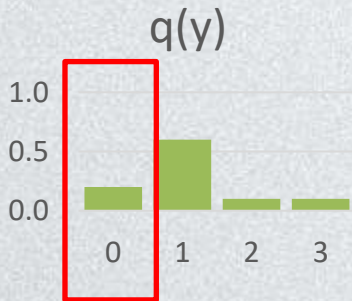


PyTorch中的损失函数

- 多分类： 假设总共有C类, $y \in \{0, 1, 2, \dots, C - 1\}$
 - $H(p, q) = -\sum_y p(y) \cdot \log q(y) = -\sum_{c=0}^{C-1} p(y=c) \cdot \log q(y=c)$



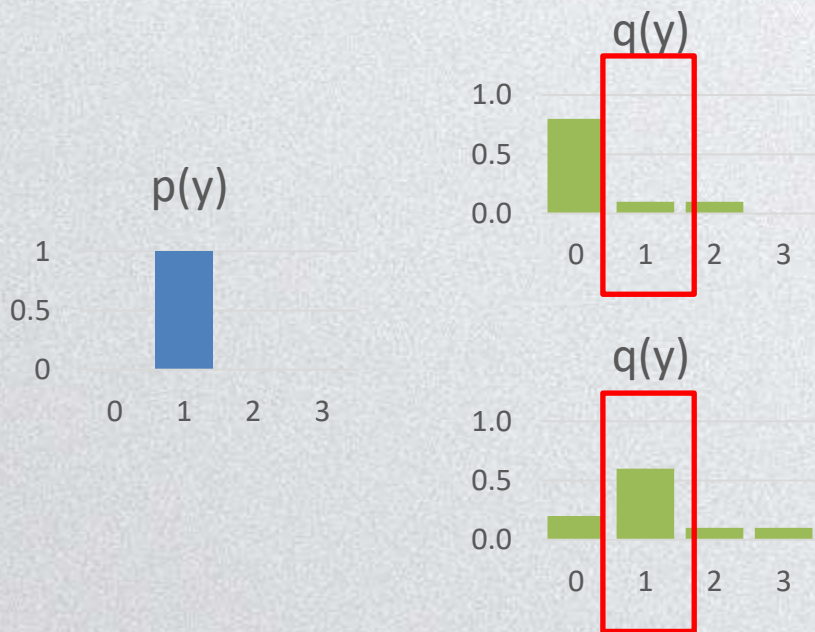
$$H(p, q) = -\log 0.8 = 0.2231$$



$$H(p, q) = -\log 0.2 = 1.6094$$

PyTorch中的损失函数

- 多分类：假设总共有C类, $y \in \{0, 1, 2, \dots, C - 1\}$
 - $H(p, q) = -\sum_y p(y) \cdot \log q(y) = -\sum_{c=0}^{C-1} p(y=c) \cdot \log q(y=c)$



$$H(p, q) = -\log 0.1 = 2.3026$$

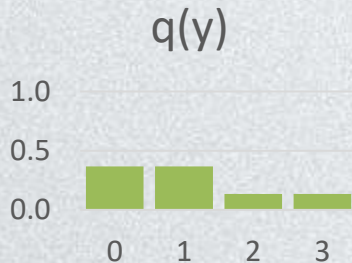
$$H(p, q) = -\log 0.6 = 0.5108$$

PyTorch中的损失函数

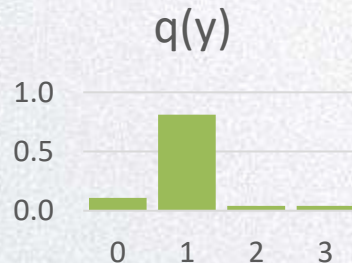
- 多分类： 假设总共有C类, $y \in \{0, 1, 2, \dots, C - 1\}$
 - $H(p, q) = -\sum_y p(y) \cdot \log q(y) = \sum_{y=0}^{C-1} p(y) \cdot \log q(y)$
- 问题： 如何得到 $q(y)$?
 - 模型输出一个C维向量 $\mathbf{z} = [z[0], z[1], \dots, z[C - 1]] \in R^C$
 - 利用softmax函数计算:

$$q(y = i) = \frac{e^{z[i]}}{\sum_j e^{z[j]}}$$

y	z	exp(z)	q(y)
0	1.000	2.718	0.366
1	1.000	2.718	0.366
2	0.000	1.000	0.134
3	0.000	1.000	0.134



y	z	exp(z)	q(y)
0	1.000	2.718	0.110
1	3.000	20.086	0.810
2	0.000	1.000	0.040
3	0.000	1.000	0.040





PyTorch中的损失函数

- PyTorch提供的交叉熵损失函数：
 - [nn.CrossEntropyLoss](#)
 - 输入：
 - $N \times C$ 维矩阵 $\mathbf{Z}$, 其中每一行为 $\mathbf{z} = [z[0], z[1], \dots, z[C - 1]] \in \mathbb{R}^C$
 - N 维向量 $\mathbf{y}$, 其中每个元素为 $y \in \{0, 1, 2, \dots, C - 1\}$
 - 注意：该损失函数同时计算了softmax函数和交叉熵函数：

$$\text{loss}(\mathbf{Z}, \mathbf{y}) = - \sum_{i=0}^{N-1} \log\left(\frac{e^{\mathbf{Z}[i, \mathbf{y}[i]]}}{\sum_j e^{\mathbf{Z}[i, j]}}\right)$$



PyTorch中的损失函数

- PyTorch提供的交叉熵损失函数 **(例子)** :
 - [nn.CrossEntropyLoss](#)
 - 输入:
 - $N \times C$ 维矩阵 $\mathbf{z}$, 其中每一行为 $\mathbf{z} = [z[0], z[1], \dots, z[C - 1]] \in \mathbb{R}^C$
 - N 维向量 $\mathbf{y}$, 其中每个元素为 $y \in \{0, 1, 2, \dots, C - 1\}$

```
>>> # Example of target with class indices
>>> loss = nn.CrossEntropyLoss()
>>> input = torch.randn(3, 5, requires_grad=True)
>>> target = torch.empty(3, dtype=torch.long).random_(5)
>>> output = loss(input, target)
>>> output.backward()
```


PyTorch中的优化器

- torch.optim中提供了多种优化器：
 - 在执行optimizer.step()时，优化器会根据**损失函数输出**反向传播计算的梯度，更新**模型**的参数

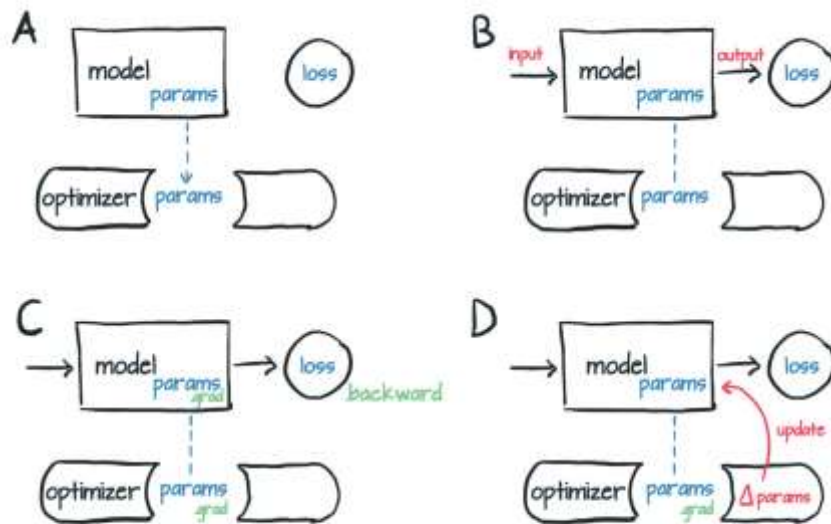


Figure 4.10 Conceptual representation of how an optimizer holds a reference to parameters (A), and after a loss is computed from inputs (B), a call to `.backward` leads to `.grad` being populated on parameter (C). At that point, the optimizer can access `.grad` and compute the parameter updates (D).

PyTorch中的优化器

- torch.optim中提供了多种优化器：
 - 不同的优化器在执行.step()时会采用不同的方式更新模型参数

```
import torch.optim as optim
dir(optim)
```

```
['ASGD',  
'Adadelata',  
'Adagrad',  
'Adam',  
'AdamW',  
'Adamax',  
'LBFGS',  
'Optimizer',  
'RMSprop',  
'Rprop',  
'SGD',  
'SparseAdam',
```



PyTorch中的优化器

- 常用的优化器：

- optim.SGD

- 随机梯度下降
 - 每次更新的学习率是固定的

$$\mathbf{w}^{t+1} = \mathbf{w}^t - lr \left(\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} \Big|_{\mathbf{w}^t} \right)^T$$

- optim.Adam

- g_t 为反向传播计算得到的梯度
 - 会在学习过程中自动的改变学习率
 - 参数不同维度间的大小差异对训练影响较小

$$\begin{cases} m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ W_{t+1} = W_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \end{cases}$$



谢谢！